P1 : 2,0
P2 : 2,5
P3 : 2,5
P4 : 2,0
~~8,5~~
9,0

Nguyễn Anh Tuấn (1)
N25DECE073.
E25CQCE02-N.

-Problem 1: Singly linked List: In Place Riversal and pointer tracing.

Given a single linked list:

head → [10] → [25] → [40] → [15] → NULL

• Task 1: Write a function algorithm in pseudocode to reserve the singly linked list, including three pointer (prev, curr, next) to complete your function.

• Task 2: Draw a diagram or step by step pointer trace table showing the position and value of the pointer (prev, curr, next) after each iteration until the list is fully reversed.

-Problem 2: Circular linked list - Insert at Head

Given a singly circular linked list:

head → [10] → [20] → [30] → (point back to head

• Task 1: Write pseudocode for function InsertHead (Head, NewValue) to insert a new node, containing new Value at the front of the

Quang Tâm

Scanned with CamScanner

circular linked list while maintaining its circular structure

• task 2: Trace the execution steps (using text description or pointer diagrams) when Inserting newValue = 5 into the given list

Problem 3: Doubly linked list - Insert before Target and Edge ~~class~~ cases.

Given a doubly linked list containing integers

• Task 1: Write pseudocode for InsertBefore (head, target, newValue) to insert a new node containing newValue immediately ~~before~~ before the ~~first~~ first ~~occurrence~~ occurrence of target.

• task 2: List the exact order of pointer assignment required and briefly ~~justify~~ justify ~~why~~ why this specific sequence avoids memory ~~leak~~ leaks vs broken links.

• task 3: Explicitly handle the following two edge cases in code/pseudocode:

1: target is located at the head node

2: target doesn't exist in the list.

Quang Tâm

Scanned with CamScanner

Problem A: Sorted linked list - Insertion.

Given a singly linked list sorted in ascending order: head → [2] → [5] → [8] → NULL

- task 1: write the function sortedInsert(head, val) to insert val into the list such that the ascending order is preserved.
- task 2: Construct a step by step trace table detailing pointer iterations and condition evaluations when inserting val = 6 into the list.

Answer

Problem 1:

task 1: Pseudocode

```
function ReverseList(head):
    prev = NULL
    curr = head
    next = NULL
    While (curr is not NULL)
        next = curr.next
        curr.next = prev
```


Scanned with CamScanner

prev = curr

curr = next

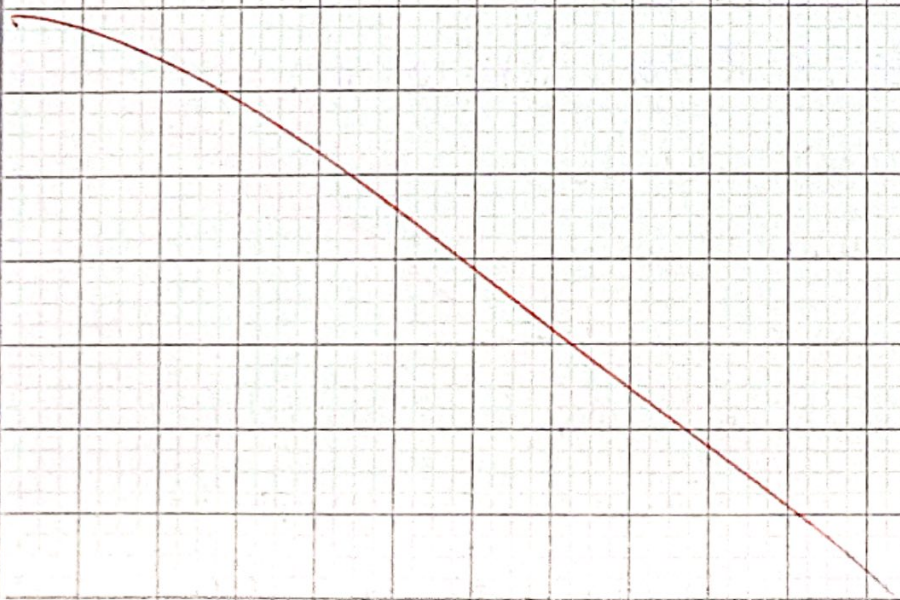
head = prev

return head.

6,75 Task 2: Trace Table.

Initialization	NULL	[10]	NULL	Linked Sharing
iteration 1	[10]	[25]	[25]	?
iteration 2	[25]	[40]	[40]	
iteration 3	[40]	[15]	[15]	
iteration 4	[15]	NULL	NULL	

Final result: ?



Scanned with CamScanner

Nguyễn Anh Tuấn (2)

3.5 Problem 2:

Task 1: Pseudo code

function InsertHead(head, newValue):

newNode = create Node(newValue)

// If list is empty.

if head is NULL

newNode.next = newNode

head = newNode

return head

// If list is not empty.

newNode.next = head

temp = head

// Traverse to find the last node

While temp.next != head:

temp = temp.next

// Connect last node to the new head

temp.next = newNode

head = newNode

return head

Tin học Văn phòng Quang Tâm

Scanned with CamScanner

1.5 Task 2 : Execution steps . (NewValue = 5)

Given list :

head → [10] → [20] → [30] → (point back to

1. Create newNode with value 6

2. Set newNode.next = head (5 points to [10])

3. Initialize temp = head ([10]), traverse until ([10]) and traverse until

temp.next == head . temp .

moves from [10] → [20] → [30] . temp stop at [30] .

4 : Set temp.next = newNode

([30] now point to 5 instead of [10])

5 : Update head = newNode

(head now points to 5)

final list .

head → [5] → [10] → [20] → [30] → (point back to head)

2.5 Problem 3 :

1.75 task 1 an task 3 :

function InsertBefore (head, target, newValue):

// Edge case 2 (part A) : List is Empty

Scanned with CamScanner

```
if head is NULL:
    return head
newNode = createNode(newValue)
// Edge Case 1: Target is at the head node
if head.value == target:
    newNode = createNode(newNode)
    newNode.next = head
    newNode.prev = NULL
    head.prev = newNod
    head = newNode
    head = newNode
    return head
curr = head
// Traverse to find the target.
while is Not NULL and curr.value != target
    curr = curr.next.
// Edge Case 2 (Part B): Target does not exist in the list
if curr is NULL
    return head // Do nothing, return original list
```

Quang Tâm

Scanned with CamScanner

// standard insertion before 'curr'
newNode.next = curr
newNode.prev = curr.prev
curr.prev.next = ~~Node~~ newNode
curr.prev = NewNode
Return head

0,75 Task 2:
1 ~~Next~~ newNode.next = curr
newNode.prev = curr.prev
curr.prev.next = newNode
curr.prev = newNode

2.0 Problem 4:
0,75 Task 1:
function SortedInsert(head, val):
newNode = new Node(val)
// Handle empty list or insertion at the head
if head == NULL or head.val >= val:
newNode.next = head
return *newNod

Quang Tâm
Scanned with CamScanner

Nguyễn Anh Tuấn (3)

// Traverse to find the correct ~~or~~ insertion point curr head

While (curr.next != NULL and ~~curr.next~~ curr.next.val < val:)

curr = curr.next

// Insert newNode between curr and curr.next

newNode.next = curr.next

curr.next = newNode

return head

1,25 Task 2:

Step	curr Position	Condition. Action. curr.next != NULL and curr.next.val < 6
Init	Node [2]	curr.next is [5]. 5 < 6 (True) curr = curr.next.
~~Iter~~ Iter	Node [5]	curr.next is [8]. 8 < 6 (F) exitLoop
Insert	Node [5]	N/A newNode.next = curr.next curr.next = newNode

Quang Tâm

Scanned with CamScanner

Final list: head -> [2] -> [5] -> [~~6~~ 6]
-> [8] -> NULL

Quang Tâm

Scanned with CamScanner